



Rappels SQL, PostgreSQL & PostGIS

Master II GéoNum, Promotion 2026

Université Lyon – Campus Porte des Alpes
24/09/2026

Bastien CAHIER – MCP Data Solutions



01 module de 3H : 24 septembre.

1. requêtes de sélection

- A. Simples
- B. Agrégats
- C. Jointures attributaires
- D. Jointures spatiales
- E. Requêtes imbriquées

2. Requêtes d'insertion, mise à jour et suppression

- A. Insertions
- B. Mise à jour
- C. Suppression



- Se rendre à l'adresse gitHub :
https://github.com/bcahier/cours_m2_sql_2027.git
- Télécharger l'archive du repository (ou cloner le git...),
- Décompresser l'archive,
- Suivre la procédure du fichier importer_la_bdd.md

Base de données du projet géonumérique CUMUL'O (Promotion 2019/2020) (L. Gonon, B. Cahier, D. Alindawa, B. Blanc) (sources : Grand Lyon / insee 2020)

- table *capteurs* : **nom, adresse, et position** des capteurs de précipitations de la métropole du Grand Lyon
- table *relevés* : relevés de précipitation mensuelle (mm) pour chaque capteur
- table *pop_200* : statistiques de la population par carreau INSEE
- table *communes_metrop* : communes de la métropole de Lyon.

I. Les requêtes de sélection

```
SELECT champ(s) FROM tablename [WHERE condition(s)] [ORDER BY champ(s)]  
[LIMIT value];
```

1. sélectionner les capteurs (table *capteurs*, champs *nom* et *adresse*) positionnés dans la ville de **Lyon** (champ *adresse*), et triés par *nom* (Z-A)
2. Sélectionner le nombre de carreaux de la métropole (table *pop_200*) ayant une proportion de ménages en maison (*men_mais*) > **85%** **OU** < **15 %**
(total = ménages collectifs *men_coll* + ménages en maison *men_mais*)
3. Requête geo Sélectionner le nom, latitude et longitude de la commune (table *communes_metrop*, champs *nom* & *geom*) dont le centre est à le plus proche à l'Est du 5^e méridien.

```
SELECT champ(s) FROM tablename [WHERE condition(s)] [ORDER BY champ(s)]  
[LIMIT value];
```

1. sélectionner les capteurs (table *capteurs*, champs *nom* et *adresse*) positionnés dans la ville de Lyon (champ *adresse*), et triés par *nom* (Z-A)

```
SELECT nom, adresse FROM capteurs WHERE adresse ILIKE '%lyon%' ORDER BY nom desc;
```

Info : ILIKE permet de s'affranchir de la casse (insensible MAJUSCULE/minuscule)

Info : Possibilité d'utiliser les REGEX avec ~ OU ~* (sensible ou non à la casse)

```
SELECT champ(s) FROM tablename [WHERE condition(s)] [ORDER BY champ(s)]  
[LIMIT value];
```

2. Sélectionner le nombre de carreaux de la métropole (table *pop_200*) ayant une proportion de ménages en maison (*men_mais*) > 85% OU < 15 % (total = ménages collectifs *men_coll* + ménages en maison *men_mais*)

```
SELECT count(*) AS nb_carreaux FROM pop_200 WHERE (( men_mais * 100/ (men_mais +  
men_coll) ) > 85 )  
OR ( (men_mais*100/(men_mais + men_coll)) < 15);
```

OU BIEN

```
SELECT count(*) AS nb_carreaux FROM pop_200 WHERE ( men_mais * 100/ (men_mais +  
men_coll) ) NOT BETWEEN 15 AND 85;
```

```
SELECT champ(s) FROM tablename [WHERE condition(s)] [ORDER BY champ(s)]  
[LIMIT value];
```

3. Sélectionner le nom, latitude et longitude de la commune (table *communes_metrop*, champs *nom* & *geom*) dont le centre est à le plus proche à l'Est du 5^e méridien.

```
SELECT nom, st_x(st_centroid(geom)) as longitude, st_y(st_centroid(geom)) as latitude  
FROM communes_metrop WHERE st_x(st_centroid(geom)) > 5 ORDER BY longitude  
LIMIT 1;
```

st_centroid(geom) : récupère le centroïde de la géométrie

st_x(geom) / *st_y(geom)* : récupère la longitude / la latitude du point

Fonctions communes d'agrégat:

- `min(col)`, `max(col)`, `sum(col)`, `avg(col)`
- `ST_collect(géométrie)`, `ST_UNION(géométrie)`
- `string_agg(col, séparateur)`, `array_agg(col)`

→ **Peuvent être combinées avec la clause GROUP BY champ1, champ2**

[pour aller plus loin](#)

1. Sélectionner la moyenne des précipitations, regroupées par mois pour l'année 2018 pour toute la métropole.
table *releves*, colonnes *precipitation*, *mois*, *annee*

```
SELECT avg(precipitation) as precip_moy, mois FROM releves WHERE annee = 2018 GROUP  
BY mois ORDER BY mois;
```

Pour info

1bis : Comment extraire le mois, l'année depuis une date ? (on dispose d'un champ *date_mois* au format date)

```
SELECT date_part('YEAR', date_mois) as annee, date_part('MONTH', date_mois) as mois  
FROM releves;
```

1ter : Comment afficher le nom du mois ? (on dispose d'un champ *date_mois* au format date)

```
SELECT to_char(date_mois, 'MONTH') FROM releves;
```

Fonctions communes d'agrégat:

- **min(col), max(col), sum(col), avg(col)**
- **ST_collect(géométrie), ST_UNION(géométrie)**
- **string_agg(col, separateur), array_agg(col)**

→ **Peuvent être combinées avec la clause GROUP BY champ1, champ2, champX**

[pour aller plus loin](#)

2. Fusionner en une seule géométrie les communes dont le nom contient « MONT-D'OR »
table *communes_metrop*, champs *nom*, *geom*

Fonctions communes d'agrégat:

- **min(col), max(col), sum(col), avg(col)**
- **ST_collect(géométrie), ST_UNION(géométrie)**
- **string_agg(col, séparateur), array_agg(col)**

→ **Peuvent être combinées avec la clause GROUP BY champ1, champ2**

[pour aller plus loin](#)

2. Fusionner en une seule géométrie les communes dont le nom contient « MONT-D'OR »
table *communes_metrop*, champs *nom*, *geom*

```
SELECT string_agg(nom, ' / ') as noms, st_union(geom) as geom FROM communes_metrop  
WHERE nom LIKE '%MONT-D''OR%';
```

Trois manières de faire une requête attributaire entre plusieurs tables :

1. Jointure implicite

```
select * from table1, table2 where table1.id = table2.id and conditions;
```

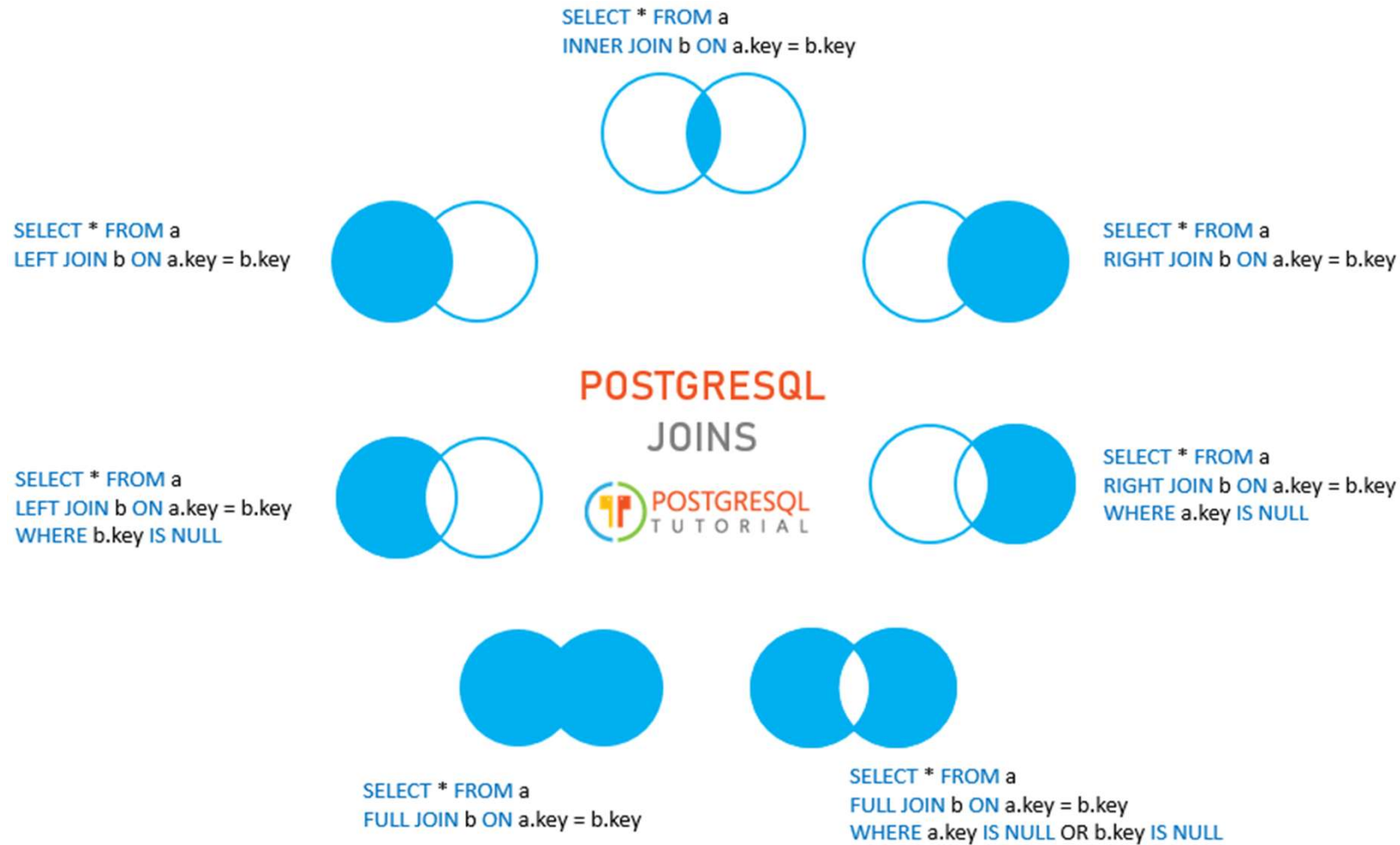
2. Jointure explicite

```
select * from table1 JOIN table2 ON table1.id = table2.id where condition;
```

3. Jointure avec une sous-requête

```
select * from table1 where table1.id IN (SELECT id from table2  
                                         WHERE condition);
```

Les requêtes de sélections – Jointures attributaires - exemples





1. Afficher les noms des capteurs et les précipitations enregistrées en **août 2017**, les trier par volume de précipitation décroissant.
2. Un capteur n'a pas de précipitations associées, de quel capteur s'agit-il?
3. Afficher les cumuls de précipitation annuels pour le capteur « Villeurbanne », dans l'ordre chronologique

1. Afficher les noms des capteurs et les précipitations enregistrées en **août 2017**, les trier par volume de précipitation décroissant.

```
SELECT capteurs.nom, releves.*  
FROM capteurs INNER JOIN releves  
ON releves.id_capteur = capteurs.id  
WHERE annee = 2017 AND mois = 8  
ORDER BY releves.precipitation desc;
```

2. Un capteur n'a pas de précipitations associées, de quel capteur s'agit-il?

```
SELECT capteurs.nom FROM capteurs LEFT JOIN releves  
ON capteurs.id = releves.id_capteur  
WHERE releves.id_capteur is null;
```

3. Afficher les cumuls de précipitation annuels pour le capteur « Villeurbanne », dans l'ordre chronologique

```
SELECT capteurs.nom, sum(precipitation) as cumul, annee  
FROM capteurs INNER JOIN releves  
ON capteurs.id = releves.id_capteur  
WHERE capteurs.nom = 'Villeurbanne'  
GROUP BY annee, nom  
ORDER BY annee asc;
```

fonctions communes de jointure spatiale (PostGIS):

- **ST_intersects**(geom1, geom2) : geom1 et geom2 s'intersectent
- **ST_Touches**(geom1, geom2) : les deux géométries se touchent sans se croiser
- **ST_Contains**(geom1, geom2) : la géométrie 2 est contenue dans la géométrie 1
- **ST_Overlaps**(geom1, geom2) : les deux géométries se chevauchent sans se contenir

1. Afficher, pour chaque capteur, sa commune d'appartenance et sa position (adresse, coordonnées)

2. Pour chaque commune, donner son nombre de ménage (champ men de la table pop_200), **Attention au piège.**

fonctions communes de jointure spatiale (PostGIS):

- **ST_intersects**(geom1, geom2) : geom1 et geom2 s'intersectent
- **ST_Touches**(geom1, geom2) : les deux géométries se touchent sans se croiser
- **ST_Contains**(geom1, geom2) : la géométrie 2 est contenue dans la géométrie 1
- **ST_Overlaps**(geom1, geom2) : les deux géométries se chevauchent sans se contenir

1. Afficher, pour chaque capteur, sa commune d'appartenance et sa position (adresse, coordonnées)

```
SELECT comm.nom as commune, capteurs.adresse as adresse, y, x FROM communes_metrop  
comm INNER JOIN capteurs ON ST_CONTAINS(comm.geom,capteurs.geom);
```

...Qui donne le même résultat que:

```
SELECT comm.nom as commune, capteurs.adresse as adresse, y, x FROM communes_metrop  
comm, capteurs WHERE ST_CONTAINS(comm.geom,capteurs.geom);
```

fonctions communes de jointure spatiale (PostGIS):

- **ST_intersects**(geom1, geom2) : geom1 et geom2 s'intersectent
- **ST_Touches**(geom1, geom2) : les deux géométries se touchent sans se croiser
- **ST_Contains**(geom1, geom2) : la géométrie 2 est contenue dans la géométrie 1
- **ST_Overlaps**(geom1, geom2) : les deux géométries se chevauchent sans se contenir

2. Pour chaque commune, donner son nombre de ménage (champ men de la table pop_200), **Attention au piège.**

```
SELECT comm.nom, sum(men) as nb_menages FROM communes_metrop comm JOIN pop_200 ON  
st_intersects(comm.geom,pop_200.geom) group by comm.nom;  
ne marche pas, pensez à la projection! intersection uniquement entre deux SRID identiques
```

```
Donc... SELECT comm.nom, sum(men) as nb_menages FROM communes_metrop comm join  
pop_200 ON st_intersects(comm.geom,st_transform(pop_200.geom,4326)) GROUP BY  
comm.nom;
```

Requête imbriquée : une requête dans une autre requête. Utile pour des requêtes plus complexes : calculs, jointure entre tables, etc...

Exemple `SELECT * from tableA where champ1 = (SELECT max(champ2) FROM tableB)`

1. Sélectionner le relevé pour lequel a été enregistré la précipitation maximale
2. Pour aller plus loin, trouver le capteur associé à ce relevé

Requête imbriquée : une requête dans une autre requête. Utile pour des requêtes plus complexes : calculs, jointure entre tables, etc...

Exemple `SELECT * from tableA where champ1 = (SELECT max(champ2) FROM tableB)`

1. Sélectionner le relevé pour lequel a été enregistré la précipitation maximale

```
SELECT * from releves WHERE precipitation = (select max(precipitation) from  
releves);
```

... Donne le même résultat que:

```
SELECT * from releves ORDER BY precipitation desc LIMIT 1;
```

Requête imbriquée : une requête dans une autre requête. Utile pour des requêtes plus complexes : calculs, jointure entre tables, etc...

Exemple `SELECT * from tableA where champ1 = (SELECT max(champ2) FROM tableB)`

2. Pour aller plus loin, trouver le capteur associé à ce relevé

```
SELECT capteurs.* , maxi.precipitation FROM capteurs INNER JOIN (  
    SELECT * FROM releves WHERE precipitation = (SELECT max(precipitation) FROM  
    releves)) as maxi  
ON capteurs.id = maxi.id_capteur;
```

Alternative, utilisation d'un WITH :

```
with maxi as (SELECT * FROM releves WHERE precipitation = (SELECT max(precipitation)  
FROM releves))  
SELECT capteurs.*, maxi.precipitation FROM capteurs INNER JOIN maxi ON capteurs.id =  
maxi.id_capteur;
```

II. Les requêtes d'insertion, mise à jour et suppression

INSERT INTO table1 (champ1, champ2, champ3) VALUES (1, 'a', true)[,(2,'b',false),...];

1. Insérer un nouveau capteur dans la table capteur, aux coordonnées 46.1 Nord, 4.12 Est

```
INSERT INTO capteurs (nom, adresse, x,y,geom) VALUES ('new_capteur', 'adresse  
inconnue', 4.12, 46.1, ST_GeomFromText('point(4.12 46.1)', 4326));
```

Pour insérer une nouvelle géométrie, plusieurs possibilités:

- ST_geomFromText('POINT(lon lat)', EPSG) → implique du texte (ex 'POINT(4.5 12.54)')
- ST_SetSRID(ST_makePoint(lon, lat), EPSG) → n'implique pas de texte

Exemples d'EPSG : 2154, 4326...

UPDATE table1 SET champ1 = valeur WHERE condition

1. Affecter 'UNK' aux trigrammes des communes de la métropole qui ne sont pas définis.
2. Changer la projection des géométries de la table pop_200, passer en WGS 84
 - **ST_srid(geom)** pour connaître la projection actuelle
 - **ST_transform(geom, EPSG)** pour reprojeter avec un nouvel EPSG (WGS 84 = 4326)

Pour aller plus loin, augmenter de 10 les précipitations associées au capteur « Bron »

UPDATE table1 SET champ1 = valeur WHERE condition

1. Affecter 'UNK' aux trigrammes des communes de la métropole qui ne sont pas définis.

```
UPDATE communes_metrop SET trigramme = 'UNK' WHERE trigramme is null;
```

UPDATE table1 SET champ1 = valeur WHERE condition

2. Changer la projection des géométries de la table pop_200, passer en WGS 84

- **ST_srid(geom)** pour connaître la projection actuelle
- **ST_transform(geom, EPSG)** pour reprojeter avec un nouvel EPSG (WGS 84 = 4326)

```
UPDATE pop_200 SET geom = st_transform(geom,4326);
```

UPDATE table1 **SET** champ1 = valeur **WHERE** condition

Pour aller plus loin, augmenter de 10 les précipitations associées au capteur « Bron »

```
UPDATE releves SET precipitation = precipitation+10  
WHERE id_capteur = (SELECT id from capteurs WHERE nom = 'Bron');
```

DELETE FROM table1 WHERE condition

1. Supprimer les capteurs situés dans la ville de Lyon.

2. Approche difficile : supprimer les capteurs localisés (se baser sur la géométrie) sur la commune de Bron

DELETE FROM table1 WHERE condition

1. Supprimer les capteurs situés dans la ville de Lyon.

```
DELETE from capteurs WHERE adresse like '%LYON%';
```

DELETE FROM table1 WHERE condition

2. Approche difficile : supprimer les capteurs localisés (se baser sur la géométrie) sur la commune de Bron

```
DELETE FROM capteurs WHERE id IN (  
    SELECT capteurs.id FROM capteurs  
    INNER JOIN communes_metrop ON st_contains(communes_metrop.geom,  
capteurs.geom)          WHERE communes_metrop.nom = 'BRON');
```



Merci de votre attention

Questions ?